§ 10. Общие методы персистентности и дерево версий

В предыдущих параграфах мы знали форму структуры и могли явно выписать изменяемый путь. Общий метод должен работать на уровне полей и ссылок: сохранить старое значение, найти нужное состояние при чтении и не потерять связи после копирования узла. Его стоимость зависит от модели памяти и вида истории.

Начнём с полностью проверяемого частного случая — истории одной ячейки на линейной шкале времени. Затем рассмотрим копирование узлов с ограниченным числом входящих ссылок, объясним, что меняется при ветвлении, и построим эйлерову нумерацию дерева версий. Завершим точным форматом сериализации и десериализации корневого упорядоченного дерева. Нумерация готового дерева и поддержание порядка в растущем дереве — разные задачи.

10.1. Сначала фиксируем модель вычислений

Частичная персистентность разрешает читать любую старую версию, но изменять только последнюю. История изменений линейна. Полная персистентность разрешает продолжить любую версию; если каждое изменение имеет одного родителя, история — дерево. Конфлюэнтные операции объединяют несколько версий и дают граф зависимостей. Эйлерова техника этой главы рассчитана на дерево, а не на произвольный граф.

Функциональность — свойство реализации: опубликованные объекты не изменяются. Оно не тождественно одной из трёх форм истории. Неизменяемые узлы могут представлять как линейную историю, так и ветвящуюся, а при аккуратно определённом слиянии — конфлюэнтную. Напротив, частичная персистентность может использовать изменяемые служебные журналы.

Считаем, что каждый узел имеет постоянное число полей фиксированного размера. Чтение одного поля, запись одного поля и работа с указателем занимают $O(1)$. Нужно отдельно указать, ограничено ли число входящих ссылок на узел. Постоянное число детей не означает постоянного числа родителей в общем графе.

Есть ещё два графа, которые нельзя смешивать. Дерево версий показывает происхождение состояний. Граф представления показывает, какие физические узлы используются корнями. Даже когда история является деревом, представление обычно содержит разделяемые узлы и не является деревом.

Редакторский пример. Версии v_1 и v_2 продолжают v_0 разными изменениями. В графе истории у них один общий предок. В памяти они могут разделять десятки поддеревьев, а каждое поддерево может иметь ссылки от нескольких разных корней. Номер версии не заменяет адрес узла и наоборот.

10.2. Полное копирование как эталон

Самый простой метод сохраняет полную копию состояния перед изменением. Для массива длины n это $O(n)$ времени и дополнительной памяти на версию. Чтение не требует поиска истории: выбираем копию и выполняем обычный запрос. При q изменениях получается $O(nq)$ памяти.

Этот способ полезен как тестовая модель и иногда как рабочее решение при очень маленьком состоянии. Его нельзя отвергать только из-за асимптотики: если состояние содержит десять чисел, дополнительная сложность графа узлов может не окупиться. Но при миллионе элементов и точечных изменениях полное копирование расходует слишком много памяти.

Копирование пути уменьшает работу до числа предков изменяемого узла. Для дерева отрезков это логарифм, для списка — расстояние от головы, для несбалансированного BST — его высота. Метод хорош, когда путь известен и все ссылки на изменяемую часть можно восстановить через новый корень.

(c) Aleksey Tolstikov

В произвольном графе у узла может быть много родителей. Простая попытка «скопировать всех родителей» может затронуть большую часть структуры или зациклиться при наличии циклов. Поэтому общий результат требует более сильных предпосылок, чем корректность конкретного дерева.

10.3. Толстая ячейка на линейной шкале времени

Вместо копирования объекта сохраним историю значения: пары `(time,value)` с возрастающими временами. Чтение на момент t выбирает последнюю запись с временем не больше t. Отсутствие новой записи между двумя моментами означает, что значение не менялось.

Возьмём начальное значение 4 в момент 0, затем записи `(3,9)` и `(8,-1)`. Чтение в моменты 0, 2, 3, 7, 8 даёт соответственно 4, 4, 9, 9, -1. Запрос в момент 100 тоже вернёт -1: в контракте этой ячейки значение действует до следующей записи. Контроль существования глобального номера версии остаётся обязанностью вызывающего кода.

```
class HistoricalCell {
    std::vector<std::pair<int,long long>> history;
public:
    explicit HistoricalCell(long long initial):history{{0,initial}} {}
    void write(int time,long long value) {
        if (time<=history.back().first)
            throw std::invalid_argument("time must increase");
        history.emplace_back(time,value);
    }
    long long read(int time) const {
        if (time<0) throw std::out_of_range("negative time");
        auto it=std::upper_bound(history.begin(),history.end(),time,
            [](int t,const auto& entry){ return t<entry.first; });
        return std::prev(it)->second;
    }
};
```

`upper_bound` находит первую запись с временем строго больше запроса; предыдущая содержит ответ. Начальная запись в момент 0 гарантирует, что для неотрицательного времени предшественник существует. Проверка возрастающих времён исключает случай, когда поздняя запись вставляется в середину уже опубликованной истории.

Если у поля было m изменений, чтение занимает $O(\log(m+1))$, хранение — $O(m+1)$. Дописывание в `vector` занимает амортизированное $O(1)$, а отдельное расширение массива может стоить $O(m)$. Для жёсткой гарантии добавления требуется другое хранилище. Время здесь относится к физической линейной истории самого журнала: мы не ветвим его обновления.

Толстый узел хранит такие истории для своих полей. Если обычная операция читала k полей, после преобразования она может потребовать $O(k \log q)$ времени. Это простой общий способ сохранения прошлого, но он не даёт автоматически постоянной надбавки на чтение.

10.4. Ограниченная коробка изменений и копирование узла

Чтобы не искать в длинном журнале, можно хранить в узле исходные поля и небольшую коробку последних модификаций. Для частичной персистентности каждая запись содержит поле, время и новое значение. При чтении старой версии проверяем постоянное число записей и выбираем подходящее состояние. Если коробка ещё не заполнена, изменение дописывается в неё.

Когда коробка заполнена, создаём копию узла в его текущем состоянии и применяем новое изменение к копии. Старый узел остаётся доступен старым версиям. Текущие входящие ссылки нужно

перенаправить на копию. Каждое перенаправление само является изменением поля родителя и тоже может вызвать копирование.

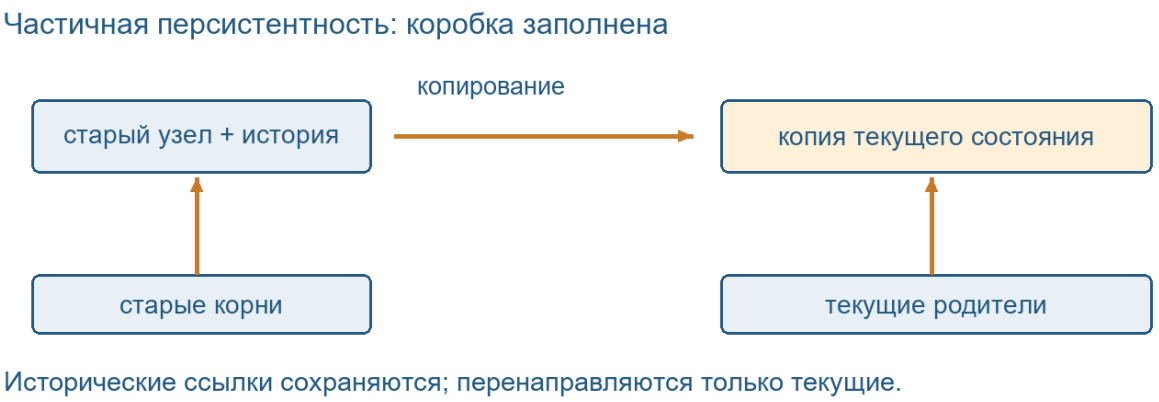


Рисунок 1: Старая версия читает прежний узел и его историю; текущие ссылки переводятся на новую копию.

Важна формулировка «текущие входящие ссылки». Старые корни и исторические связи не переписываются. Для поиска текущих родителей обычно поддерживаются обратные сведения. Если на узел в текущем состоянии указывают p родителей и p не ограничено, одна его копия потребует $\Omega(p)$ перенаправлений. Именно поэтому условие ограниченной входящей степени существенно.

В модели указателей с постоянным числом полей и ограниченной входящей степенью существуют стандартные конструкции частичной персистентности с постоянной амортизированной дополнительной памятью на элементарную модификацию и постоянной стоимостью доступа к полю. Реализация должна правильно выбрать ёмкость коробки, поддерживать текущие обратные ссылки и учитывать каскады копирования. Это утверждение о такой конструкции, а не о любом произвольном «журнале плюс копирование».

Интуиция учёта следующая. Заполненные коробки накапливают выполненные модификации; создание копии освобождает запас места для будущих изменений. Каскад не бесплатен, но его стоимость оплачивается ранее заполненными коробками. Потенциал учитывает занятые места, а постоянный предел входящих ссылок ограничивает число новых перенаправлений от одной копии. Без этих условий нельзя завершить амортизационный аргумент.

В этой главе данный результат используется как граница применимости общего метода. Полная реализация с обратными указателями и полное доказательство теоремы не заменяются несколькими строками псевдокода. Практические листинги главы реализуют более простые и явно ограниченные интерфейсы: линейную историческую ячейку и работу с готовым деревом.

10.5. Почему время создания не описывает ветвление

Пусть в v_0 поле равно 0. Версия v_1 продолжает v_0 и записывает 5. Позже версия v_2 тоже продолжает v_0, но меняет другое поле. В v_2 первое поле должно остаться 0. Линейный поиск «последняя запись с номером не больше 2» ошибочно выберет запись из v_1.

Для полной персистентности модификация видна не всем более поздним версиям, а только своим потомкам, если по пути не встретилось более близкое изменение этого поля. Нужное отношение — предок/потомок, а не сравнение хронологических номеров.

(c) Aleksey Tolstikov

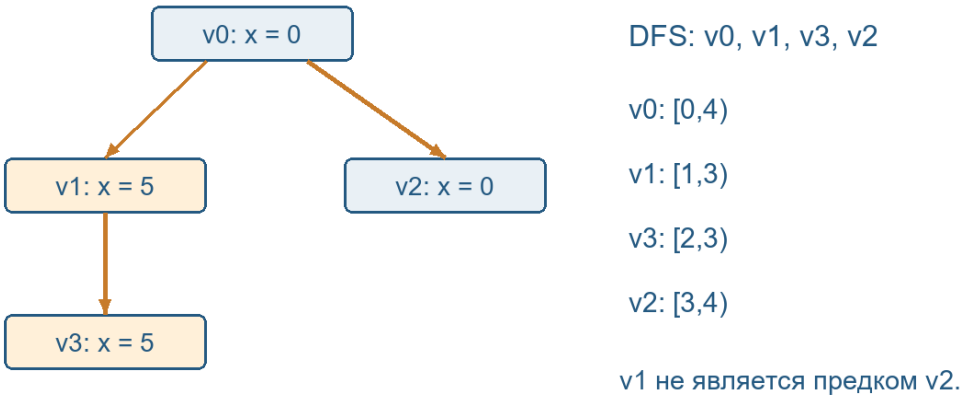


Рисунок 2: Запись в v1 видна v3, но не соседней ветви v2. Порядок создания 0,1,2,3 не кодирует это отношение.

Если для одного поля записаны модификации в нескольких вершинах дерева версий, запрос в вершине v ищет ближайшего предка v среди этих вершин. Это отдельная задача о помеченных предках. Одной проверки «является ли u предком v» недостаточно: она отвечает для выбранного кандидата, но не находит нужного кандидата среди всех записей.

Для офлайн-обработки, когда вся история заранее известна, можно выполнить DFS по дереву версий. При входе применяем изменение, отвечаем на запросы вершины, затем посещаем детей. При выходе откатываем изменение. На текущем стеке рекурсии находятся ровно предки рассматриваемой версии, поэтому значения соответствуют ей.

Это полезный способ решить задачу о запросах ко всем версиям, но не онлайн-структура полной персистентности. Он требует заранее известной истории и изменения порядка обработки. Если следующий запрос зависит от ответа предыдущего, такой перенос может быть невозможен. Откат и персистентность решают пересекающиеся, но разные задачи.

10.6. Эйлерова нумерация готового дерева

При первом посещении вершины v присвоим ей `tin[v]` и увеличим счётчик. После посещения всех детей запишем текущее значение счётчика в `tout[v]`, не увеличивая его повторно. Тогда вершины поддерева v занимают полуинтервал $[tin[v], tout[v])$.

Это одна из распространённых конвенций. Есть и другая, где счётчик увеличивается на входе и выходе. Формулы разных конвенций нельзя смешивать. В нашем варианте у листа `tout=tin+1`, а размер поддерева равен `tout-tin`.

Пусть у 0 дети 1 и 2, а у 1 ребёнок 3. При порядке детей 1, затем 2 получаем:

Вершина	tin	tout	Глубина
0	0	4	0
1	1	3	1
3	2	3	2
2	3	4	1

Условие предка, включая саму вершину:

(c) Aleksey Tolstikov

$$u \text{ — предок } v \quad \Longleftrightarrow \quad tin[u] \le tin[v] < tout[u].$$

DFS заканчивает всё поддерево до перехода к следующему брату. Поэтому между входом и выходом u посещаются ровно его потомки. Это доказывает обе стороны эквивалентности. Условие `tin[u]<=tin[v]` само по себе недостаточно: 1 посещён до 2, но не является его предком.

Рекурсивный обход прост, однако длинная цепочка может исчерпать стек. Следующая реализация использует явный стек пар «вершина, индекс следующего ребёнка». Она принимает уже проверенное дерево с корнем 0.

```
struct EulerIndex { std::vector<int> tin,tout,depth; };
// Precondition: children describes one tree rooted at 0.
inline EulerIndex index_tree(const std::vector<std::vector<int>>& children) {
    int n=static_cast<int>(children.size());
    if (!n) throw std::invalid_argument("empty tree");
    EulerIndex e{std::vector<int>(n),std::vector<int>(n),
                 std::vector<int>(n)};
    std::vector<std::pair<int,std::size_t>> stack{{0,0}};
    int timer=0;
    e.tin[0]=timer++;
    while (!stack.empty()) {
        int v=stack.back().first;
        std::size_t next=stack.back().second;
        if (next==children[v].size()) {
            e.tout[v]=timer;
            stack.pop_back();
        } else {
            ++stack.back().second;
            int u=children[v][next];
            e.depth[u]=e.depth[v]+1;
            e.tin[u]=timer++;
            stack.emplace_back(u,0);
        }
    }
    return e;
}
inline bool ancestor(const EulerIndex& e,int u,int v) {
    return e.tin[u]<=e.tin[v] && e.tin[v]<e.tout[u];
}
```

Один проход занимает $O(V)$ времени и памяти. Каждый ребёнок рассматривается один раз, каждая вершина один раз добавляется в стек и удаляется из него. Запрос предка после подготовки выполняется за $O(1)$.

Полученная нумерация позволяет использовать структуры предыдущих глав: сумма по поддереву становится запросом на отрезке эйлерова порядка. Но это не означает, что сама форма дерева может произвольно изменяться без обновления индексов. Статический порядок фиксирует конкретный обход конкретного дерева.

10.7. Динамический порядок: что нужно общему методу

Добавим нового ребёнка к вершине 1 из предыдущего примера. Его надо разместить внутри интервала поддерева 1, перед вершиной 2. Если номера `tin` — плотные целые числа, вставка сдвинет номера последующих вершин. Простое увеличение глобального счётчика даст номер в конце и нарушит свойство поддерева как интервала.

Вместо плотных чисел можно хранить события входа и выхода в упорядоченном списке. Для нового ребёнка вставляются два события перед выходом родителя. Вложенность событий сохраняется: вход родителя предшествует входу ребёнка, а выход ребёнка предшествует выходу родителя. Сравнения порядка заменяют сравнения постоянных числовых меток.

Задача поддержания порядка предоставляет `insert_after(position,item)` и `before(a,b)`. Она отличается от порядковой статистики: узнать, что стоит раньше, не то же самое, что узнать точный номер позиции. Простой двусвязный список быстро вставляет, но сравнивает произвольные элементы медленно; массив быстро сравнивает позиции, но дорого вставляет в середину.

Назначение дробных меток между соседями работает только до исчерпания точности. Повторные вставки в один промежуток потребуют всё больше битов. Метки фиксированной разрядности с редкими перенумерациями дают полезные конструкции, но частота и стоимость перенумераций требуют доказательства. Нельзя назвать такой способ $O(1)$ только потому, что среднее двух чисел вычисляется быстро.

В общих методах полной персистентности динамический порядок помогает описывать дерево версий и области действия модификаций. Однако он не решает автоматически поиск ближайшего изменения поля и перенаправление ссылок. Эти компоненты должны согласованно поддерживаться. Поэтому из наличия эйлерова обхода не следует универсальная реализация полной персистентности за постоянное время.

Для практического выбора полезно сначала спросить, нужна ли такая универсальность. Если структура — дерево отрезков, копирование пути проще и даёт явные логарифмические оценки. Если история известна заранее, обход с откатом может оказаться существенно проще общего онлайн-метода.

10.8. Сериализация: сначала однозначный формат

Эйлеровым обходом называют несколько разных последовательностей: времена входа, события входа/выхода, список вершин при каждом проходе ребра. Для восстановления дерева нужно выбрать точный формат. Одно перечисление вершин в порядке первого посещения не определяет дерево: последовательность 0,1,2 подходит и цепочке, и корню с двумя детьми.

Выберем формат для **одного корневого упорядоченного дерева без меток**. При входе в вершину пишем `(`, после всех детей — `)`. Порядок детей значим. Для одной вершины код `()`, для корня с двумя листьями — `(()())`, для цепочки из трёх вершин — `((()))`.

Число токенов равно $2V$. Непустая строка должна иметь нулевой итоговый баланс, а баланс каждого непустого собственного префикса должен быть строго положителен. Строгость исключает лес: `()()` сбалансирован, но кодирует две отдельные корневые компоненты. Также запрещены неизвестные символы и закрытие вершины при пустом стеке.

Для восстановления при `(` создаём новую вершину. Если стек непуст, добавляем её последним ребёнком текущей вершины; затем кладём её в стек. При `)` извлекаем текущую вершину. Номера присваиваются в порядке входа. Если нужны исходные метки вершин, формат должен дополнительно хранить их при открытии; текущий код восстанавливает форму и порядок, а не потерянные имена.

```cpp
// One rooted ordered tree, encoded by entry '(' and exit ')'.
inline std::vector<std::vector<int>> decode(const std::string& s) {
    if (s.empty()) throw std::invalid_argument("empty code");
    std::vector<std::vector<int>> children;
    std::vector<int> stack;
    for (char c:s) {
        if (c=='(') {
            if (stack.empty() && !children.empty())
                throw std::invalid_argument("multiple roots");
            int v=static_cast<int>(children.size());
            children.emplace_back();
            if (!stack.empty()) children[stack.back()].push_back(v);
            stack.push_back(v);
        } else if (c==')') {
            if (stack.empty()) throw std::invalid_argument("extra close");
            stack.pop_back();
        } else throw std::invalid_argument("unknown token");
    }
    if (!stack.empty()) throw std::invalid_argument("unclosed node");
    return children;
}
```

Инвариант стека: он содержит путь от корня до вершины, чья запись сейчас продолжается. Открывающая скобка создаёт ровно следующего ребёнка вершины на вершине стека. Закрывающая завершает именно этого ребёнка. Значит, любой допустимый код разбирается единственным способом. Повторная сериализация полученного дерева даст исходную строку.

Время и память равны $O(V)$. Сам текст тоже занимает $O(V)$; потоковый разбор может не хранить его целиком, но для построения списка детей всё равно нужна память под дерево. Итеративный стек позволяет обработать цепочку из сотен тысяч вершин без рекурсивных вызовов.

10.9. Проверка обхода не равна восстановлению

Проверяющий алгоритм отвечает только «допустимо или нет». Восстанавливающий возвращает структуру, на которой можно выполнять последующие запросы. Для скобочного формата проверка баланса проще построения, но её положительный ответ ещё не даёт списков детей.

В плане курса задачи 3768 «Эйлеров обход» и 4084 «Проверка ответа» указаны как мотивация. Перед применением алгоритма нужно сверить их точный формат. Если условие даёт последовательность меток при переходах по рёбрам, нельзя читать её как наши скобки. Если проверяется обход заданного дерева, одной сбалансированности тоже недостаточно: надо сопоставлять события его рёбрам и вершинам.

Проверим редакторский код `((())())`. Он задаёт корень 0 с детьми 1 и 3, у вершины 1 есть ребёнок 2. Сериализация задаёт порядок 0,1,2,3; интервалы поддеревьев равны $[0,4)$, $[1,3)$, $[2,3)$ и $[3,4)$. Теперь номера 1 и 3 являются братьями, а 1 — предок 2. Это та же форма, что на схеме дерева версий, с перенумерацией по входу.

Хорошие отрицательные тесты: пустая строка; `)(`; `((`; `())`; `()()`; строка с посторонним символом. Положительные: одна вершина, длинная цепочка, звезда, случайное упорядоченное дерево. Проверяйте круговой переход «дерево → строка → дерево» с учётом того, что исходные произвольные метки не сохраняются.

Для исторической ячейки проверьте чтение ровно на момент записи, между записями и после последней. Для дерева версий сравните `ancestor` с подъёмом по родителям на всех парах маленьких деревьев. Эти тесты проверяют реализованные интерфейсы; они не являются экспериментальным доказательством общей теоремы о персистентности.

10.10. Итоговый выбор метода

Условия задачи	Подход	Что нужно доказать
Малое состояние	полная копия	стоимость всей копии
Короткий путь изменения	копирование пути	высота и неизменность общих узлов
Линейная история полей	толстые узлы	поиск последней записи
Ограниченная входящая степень	коробки и копирование	учёт каскадов и обратных ссылок
Офлайн-дерево версий	DFS с откатом	правильный текущий путь
Онлайн-ветвление общего графа	специальный общий метод	области действия, порядок, ссылки

Общая тема всех вариантов — явно определить, какое состояние видит запрос. Оптимизация начинается после этого определения. Недостаточно сохранить несколько корней, если из них достижимы изменяемые общие данные. Недостаточно подписать изменения временем, если история ветвится. Недостаточно получить быстрые отдельные компоненты, если их совместное обновление требует полного перестроения.

Источники и границы. Глава следует разделам 3.1 и 3.4 программы и темам лабораторных по сериализации. Подтверждённый видеофрагмент 30.11.2022, 44:00–46:00, служит только связью с обсуждением памяти версий из § 7. Новая расшифровка общих методов по нему не заявляется. Для углублённого доказательства общих преобразований полезна работа Driscoll, Sarnak, Sleator, Tarjan, *Making Data Structures Persistent* (1989): она существенно шире реализованных здесь частных случаев.

10.11. Практика к главе

В расширенной подборке приложения A: №3768 «Эйлеров обход» (основная); №4084 «Проверка ответа» (основная); №3793 «Нечётный фактор» (углубление). Полные условия доступны по ссылкам в приложении. Выберите основные задачи, затем переходите к углублению; решать весь список за одну неделю не требуется.

10.12. Вопросы для самопроверки

1. Чем функциональная реализация отличается от полной персистентности?
2. Почему постоянное число детей не ограничивает число входящих ссылок?
3. Какой ответ даст историческая ячейка между двумя записями?
4. Какая часть операции `write` имеет лишь амортизированную оценку?
5. Почему при копировании узла перенаправляются только текущие ссылки?
6. Постройте контрпример к поиску записи по номеру создания в ветвящейся истории.
7. Докажите формулу предка для выбранной конвенции tin/tout.
8. Почему новый ребёнок может нарушить статическую эйлерову нумерацию?
9. Чем интерфейс поддержания порядка отличается от запроса номера позиции?
10. Почему строка ()() не задаёт одно дерево в нашем скобочном формате? Чем проверка такого формата отличается от проверки маршрута с метками вершин в задаче 4084?

(c) Aleksey Tolstikov